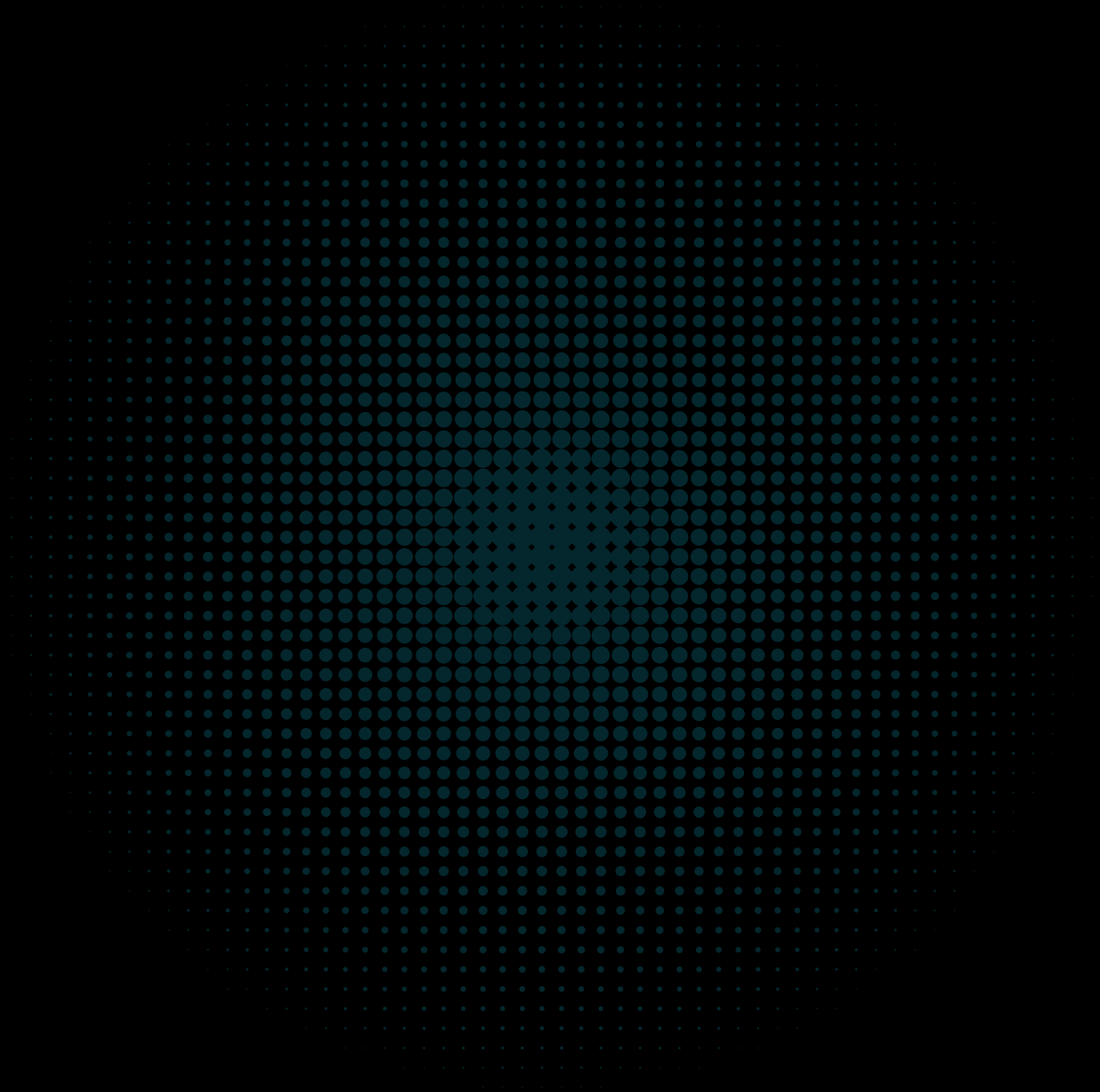




CMTAT Confidential Specification

CMTAT Functional Specifications with Zama Confidential Blockchain Protocol and ERC-7984

CMTAT Confidential version: v0.2.0
First published: April 2026



CMTAT Confidential - Confidential Security Token

A confidential security token implementation combining [CMTAT](#) compliance features with the [Zama Confidential Blockchain Protocol](#) for private balances.

Table of Contents

- [Overview](#)
- [Architecture](#)
- [Summary](#)
- [Deployment Variants](#)
- [Installation](#)
- [Compiler & EVM Version](#)
- [Security](#)
- [Roles](#)
- [Contract Functions](#)
- [Role-Based Access Control](#)
- [Troubleshooting](#)
- [References](#)

Overview

CMTAT Confidential implements the [ERC-7984](#) standard (Confidential Fungible Token) with CMTAT regulatory compliance modules. All token balances and transfer amounts are encrypted using Fully Homomorphic Encryption (FHE), ensuring transfer amount and balance privacy while maintaining regulatory compliance capabilities.

CMTAT is a security token framework by [Capital Markets and Technology Association](#) that includes various compliance features such as conditional transfer, account freeze, and token pause. The specification are blockchain agnostic with implementation available for several different blockchain ecosystem such as [Ethereum](#), [Solana](#), [Tezos](#), and [Aztec](#) (which also enables confidential transactions).

CMTAT Confidential is built on top of [OpenZeppelin Confidential Contracts](#), including its ERC-7984 implementation, and on top of the Solidity [CMTAT](#) implementation for compliance modules.

What is FHE?

Fully Homomorphic Encryption (FHE) enables computing directly on encrypted data without ever decrypting it. The Zama Protocol uses FHE combined with Multi-Party Computation (MPC) for threshold decryption and Zero-Knowledge Proofs (ZKPoKs) for input validation, providing:

- **End-to-end encryption:** Transaction inputs and state remain encrypted - no one can see the data, not even node operators
- **Composability:** Confidential contracts can interact with other contracts and dApps
- **Programmable confidentiality:** Smart contracts define who can decrypt what through the Access Control List (ACL)

Key Features

- **Confidential Balances:** All balances are stored as `euint64` (encrypted unsigned 64-bit integers) - only authorized parties can decrypt
- **Confidential Transfers:** Transfer amounts are submitted as encrypted inputs with Zero-Knowledge Proofs of Knowledge (ZKPoKs)
- **Regulatory Compliance:** Pause, freeze, and forced transfer capabilities for compliance
- **Role-Based Access Control:** Granular permissions for minting, burning, pausing, and enforcement
- **Document Management:** Attach terms, documents, and metadata to tokens
- **ERC-7984 Standard:** Based on OpenZeppelin's confidential token implementation

Architecture

```
CMTAT-Confidential
├── ERC7984 (OpenZeppelin Confidential Contracts)
│   ├── Encrypted balances (euint64)
│   ├── Confidential transfers
│   ├── Operator system
│   └── Disclosure mechanism
│
├── CMTATBaseGeneric (CMTAT Modules)
│   ├── PauseModule - Pause/unpause transfers
│   ├── EnforcementModule - Freeze/unfreeze addresses
│   ├── AccessControlModule - Role-based permissions
│   ├── DocumentEngineModule - ERC-1643 document management
│   └── ExtraInformationModule - Token metadata (tokenId, terms, info)
│
├── FHE Modules (custom extensions)
│   ├── ERC7984MintModule - Modular mint with authorization hook
│   ├── ERC7984BurnModule - Modular burn with authorization hook
│   ├── ERC7984EnforcementModule - Forced transfer and forced burn
│   └── ERC7984BalanceViewModule - Per-account balance observers (holder + role slots)
│
├── ERC7984PublishTotalSupplyModule - Public total supply disclosure (in CMTATConfidentialBase)
│   └── ERC7984TotalSupplyViewModule - Total supply observer list with auto ACL re-grant (CMTATConfidential only)
│
└── Zama Protocol Infrastructure (configured via ZamaEthereumConfig)
    ├── ACL - Access Control List for encrypted data permissions
    ├── FHEVMExecutor (Coproprocessor) - Performs FHE computations
    ├── KMSVerifier - Verifies decryption proofs from Key Management System
    └── InputVerifier - Validates encrypted inputs and ZKPoKs
```

How It Works

1. **Symbolic Execution:** When a contract calls an FHE operation, the host chain produces a pointer to the result and emits an event to notify the coprocessor network
2. **Coproprocessor Computation:** The coprocessors perform the actual FHE computation off-chain

3. **Threshold Decryption:** Decryption requests go through the Key Management Service (KMS), which uses MPC to ensure no single party can access the private key

Summary

This section maps the CMTAT framework features to the CMTAT Confidential implementation, showing how standard functionalities are adapted for Fully Homomorphic Encryption.

CMTAT Framework Mapping

CMTAT Framework Mandatory Functionalities	CMTATConfidential Corresponding Features
Know total supply	<code>confidentialTotalSupply()</code> returns <code>uint64</code> (encrypted)
Know balance	<code>confidentialBalanceOf()</code> returns <code>uint64</code> (encrypted)
Transfer tokens	<code>confidentialTransfer()</code> with encrypted amount + ZKPoK
Create tokens (mint)	<code>mint()</code> with encrypted amount + ZKPoK
Cancel tokens (burn)	<code>burn()</code> with encrypted amount + ZKPoK
Pause tokens	<code>pause()</code> - inherited from CMTAT
Unpause tokens	<code>unpause()</code> - inherited from CMTAT
Deactivate contract	<code>deactivateContract()</code> - inherited from CMTAT
Freeze	<code>setAddressFrozen(address, true)</code> - inherited from CMTAT
Unfreeze	<code>setAddressFrozen(address, false)</code> - inherited from CMTAT
Name attribute	ERC20 <code>name()</code> - public
Ticker symbol attribute	ERC20 <code>symbol()</code> - public
Token ID attribute	<code>tokenId()</code> - inherited from CMTAT
Reference to legally required documentation	<code>terms()</code> - inherited from CMTAT

Extended Features

Functionalities	CMTAT Confidential Features	Available
Forced Transfer	<code>forcedTransfer()</code> with encrypted amount	✓
Forced Burn	<code>forcedBurn()</code> with encrypted amount	✓

Functionalities	CMTAT Confidential Features	Available
Operator System	<code>setOperator()</code> / <code>confidentialTransferFrom()</code>	✓
Public Disclosure	<code>requestDiscloseEncryptedAmount()</code> / <code>discloseEncryptedAmount()</code>	✓
On-chain snapshot	Not implemented	✗
Freeze partial tokens	Not implemented (all balances are encrypted)	✗
Integrated allowlisting	Not implemented	✗
RuleEngine / transfer hook	Not implemented	✗
Upgradability	Not implemented (standalone only)	✗

Implementation Details

Functionalities	CMTAT Confidential	Note
Mint while paused	✓	Minting is allowed when contract is paused (same as CMTAT)
Burn while paused	✓	Burning is allowed when contract is paused (same as CMTAT)
Self burn	✗	Only <code>BURNER_ROLE</code> can burn tokens
Standard burn on frozen address	✗	Use <code>forcedBurn()</code>
Forced burn from frozen address	✓	<code>forcedBurn()</code> with <code>FORCED_OPS_ROLE</code>
Burn via <code>forcedTransfer</code>	✗	<code>forcedTransfer</code> reverts if <code>to</code> is <code>address(0)</code> -- use <code>forcedBurn()</code>
Balance overflow protection	✓	Uses FHSafeMath: transfers 0 on overflow/underflow (privacy-preserving)

Key Differences from Standard CMTAT

Aspect	CMTAT (Standard)	CMTAT Confidential (Confidential)
Balance type	<code>uint256</code> (public)	<code>euint64</code> (encrypted)
Value range	Up to $\sim 1.15 \times 10^{77}$ (<code>uint256</code>)	Up to $\sim 1.84 \times 10^{19}$ (<code>uint64</code> max = 18,446,744,073,709,551,615)

Aspect	CMTAT (Standard)	CMTAT Confidential (Confidential)
Transfer amount	<code>uint256</code> (public)	<code>externalEuint64</code> + ZKPoK
Total supply	<code>uint256</code> (public)	<code>euint64</code> (encrypted)
Balance visibility	Anyone can read	Only ACL-authorized parties can decrypt
Transfer validation	Reverts on insufficient balance	Transfers 0 silently (privacy-preserving)
Allowance system	ERC20 <code>approve</code> / <code>allowance</code>	Operator system with time-limited access
Forced Burn	Through <code>forcedTransfer</code> or <code>forcedBurn</code> if implemented	Through <code>forcedBurn</code> since the function is implemented

Important: The `euint64` type has a significantly smaller range than `uint256`. Decimals above 18 are rejected at construction (`CMTAT_DecimalTooHigh`). See [Choosing Decimals](#) for the full supply impact table.

Decryption Requirements

To decrypt encrypted values (balances, amounts, total supply), the requesting party must:

1. Have ACL permission granted via `FHE.allow()` or `FHE.allowTransient()` (verifiable with `FHE.isAllowed()` or `FHE.isSenderAllowed()`)
2. Or the value must be marked publicly decryptable via `FHE.makePubliclyDecryptable()`
3. Request decryption through the Zama Relay SDK (`@zama-fhe/relayer-sdk`)
4. Submit the decryption proof on-chain via `FHE.checkSignatures()` (reverts if the proof is invalid)

Deployment Variants

Two deployment-ready contracts are provided. Both share the same abstract base (`CMTATConfidentialBase`) and are functionally identical except for total supply visibility.

	<code>CMTATConfidential</code>	<code>CMTATConfidentialLite</code>
Confidential balances & transfers	✓	✓
Mint / Burn / Forced ops	✓	✓
Pause / Freeze	✓	✓
Per-account balance observers	✓	✓
<code>publishTotalSupply</code> (public disclosure)	✓	✓
Total supply observer list (auto ACL)	✓	✗

	CMTATConfidential	CMTATConfidentialLite
SUPPLY_OBSERVER_ROLE	✓	✓
SUPPLY_PUBLISHER_ROLE	✓	✓
Contract size	~20.5 KB	~19.2 KB

Choose `CMTATConfidentialLite` when automatic per-observer ACL re-grant on every mint/burn is not required and you want to minimize deployment cost. `publishTotalSupply` (one-shot public disclosure) is available in both variants.

Installation

```
# Clone the repository
git clone --recursive https://github.com/your-repo/CMTAT-Confidential.git
cd CMTAT-Confidential

# Install dependencies
npm install

# Compile contracts
npm run compile

# Run tests
npm run test
```

Compiler & EVM Version

- Solidity pragmas use `^0.8.27` across the codebase to stay compatible with OpenZeppelin Confidential Contracts.
- Hardhat is configured to compile with `0.8.34`.
- The configured EVM version is `prague` (see `hardhat.config.ts`).

Versioning

The contract-level `version()` string is pinned to `0.2.0` via `CMTATConfidentialVersionModule`.

Security

Automated Audit — Nethermind AuditAgent

Nethermind AuditAgent automated scan (March 18, 2026, commit `51f9d7aa`) reported **3 medium**, **2 low**, **2 info**, and **1 best practice** findings across 10 contracts (1 239 lines of code). Full rationale and fix details in [nethermind-audit-agent-report-feedback.md](#).

This report was generated entirely by AI and has not been manually reviewed by Nethermind's security team.

#	Severity	Finding	Disposition	Commit
---	----------	---------	-------------	--------

#	Severity	Finding	Disposition	Commit
1	Medium	Receiver reentrancy can bypass the <code>transferAndCall</code> rollback path	Disputed — upstream ERC-7984 design; NatSpec warning added	36dbd3f
2	Medium	Receiver reentrancy can steal tokens via <code>confidentialTransferAndCall</code>	Duplicate of #1 — resolved together	36dbd3f
3	Medium	Freezing <code>address(0)</code> gives <code>ENFORCER_ROLE</code> a global block over holder transfers	Documented — upstream fix proposed in CMTA/CMTAT#372 ; operator warning added	1abe564
4	Low	Unbounded supply-observer ACL refresh can cause OOG on mint/burn	Fixed — admin-controlled cap (<code>setMaxSupplyObservers</code> , default 10)	12249c1
5	Low	<code>forcedBurn</code> does not invoke <code>_afterBurn</code> , causing observer ACL staleness	Fixed — <code>_afterBurn</code> hook added to <code>ERC7984EnforcementModule</code>	681ebde
6	Info	<code>forcedBurn</code> does not refresh total-supply observer ACLs (full variant)	Duplicate of #5 — resolved together	681ebde
7	Info	Unbounded observer list can cause DoS on <code>mint</code> and <code>burn</code>	Duplicate of #4 — resolved together	12249c1
8	Best Practice	Duplicate observer removal via <code>setRoleObserver(account, address(0))</code>	Fixed — <code>setRoleObserver</code> now rejects <code>address(0)</code>	a74314e

Static Analysis — Aderyn

Aderyn static analysis (v0.2.0) reported **0 high** and **8 low** severity findings across 12 contracts (663 nSLOC). All findings are accepted or not applicable for this codebase. Full rationale in [aderyn-report-feedback.md](#), source report in [aderyn-report.md](#).

Command used to generate the report:

```
aderyn --output aderyn-report.md
```

ID	Finding	Instances	Disposition
L-1	Centralization Risk	11	Accepted — role-based access control is mandatory for a regulated security token
L-2	Unspecific Solidity Pragma (<code>^0.8.27</code>)	12	Accepted — lower bound required by OZ Confidential submodule; Hardhat compiles with <code>0.8.34</code>
L-3	PUSH0 Opcode	12	Not applicable — target is Ethereum mainnet, EVM version set to <code>prague</code> in <code>hardhat.config.ts</code>

ID	Finding	Instances	Disposition
L-4	Modifier Invoked Only Once	2	Accepted — consistent with the module authorization pattern across all modules
L-5	Empty Block	18	Accepted — modifier-only authorization hooks and intentional virtual extension points
L-6	Internal Function Used Only Once	1	Accepted — required by the OpenZeppelin <code>initializer</code> modifier pattern
L-7	State Change Without Event	1	Not applicable — occurs in a mock contract (<code>ConfidentialReceiverMock</code>), not production code
L-8	Unchecked Return	12	Not applicable — <code>FHE.allow()</code> / <code>FHE.makePubliclyDecryptable()</code> return the same handle (fluent interface), not an error code

Static Analysis — Slither

We attempted to run Slither with:

```
slither . --checklist --filter-paths "openzeppelin-contracts|test|forge-std" >
slither-report.md
```

But the run failed with:

```
ERROR:root:Error:
ERROR:root:The source code appears to be out of sync with the build artifacts
on disk.

    This discrepancy can occur after recent modifications to
node_modules/@fhevm/solidity/config/ZamaConfig.sol. To resolve this
    issue, consider executing the clean command of the build system (e.g.
forge clean).
ERROR:root:Please report an issue to https://github.com/crytic/slither/issues
```

At this stage, no Slither findings are available for this release due to this tooling/build-artifact sync issue.

Roles

Role	Description
<code>DEFAULT_ADMIN_ROLE</code>	Can grant/revoke all roles, deactivate contract, set total supply observer cap (<code>setMaxSupplyObservers</code>)
<code>MINTER_ROLE</code>	Can mint new tokens
<code>BURNER_ROLE</code>	Can burn tokens

Role	Description
<code>PAUSER_ROLE</code>	Can pause/unpause all transfers
<code>ENFORCER_ROLE</code>	Can freeze and unfreeze addresses — must not freeze <code>address(0)</code> (see warning below)
<code>FORCED_OPS_ROLE</code>	Can execute forced transfers and forced burns on frozen addresses
<code>OBSERVER_ROLE</code>	Can assign per-account balance observers via <code>setRoleObserver</code>
<code>SUPPLY_OBSERVER_ROLE</code>	Can manage total supply observers (<code>addTotalSupplyObserver</code> , <code>removeTotalSupplyObserver</code>)
<code>SUPPLY_PUBLISHER_ROLE</code>	Can call <code>publishTotalSupply</code>

Contract Functions

Minting

Mint tokens to an address (requires `MINTER_ROLE`):

```
// With encrypted input and proof
function mint(
    address to,
    externalEuint64 encryptedAmount,
    bytes calldata inputProof
) public onlyRole(MINTER_ROLE) returns (euint64 transferred);

// With existing encrypted handle (caller must have ACL access)
function mint(
    address to,
    euint64 amount
) public onlyRole(MINTER_ROLE) returns (euint64 transferred);
```

Burning

Burn tokens from an address (requires `BURNER_ROLE`):

```
function burn(
    address from,
    externalEuint64 encryptedAmount,
    bytes calldata inputProof
) public onlyRole(BURNER_ROLE) returns (euint64 transferred);
```

Transfers

ERC-7984 exposes eight transfer function variants:

Function	Description
----------	-------------

Function	Description
<code>confidentialTransfer(to, amount)</code>	Transfer using existing handle
<code>confidentialTransfer(to, amount, proof)</code>	Transfer with encrypted input
<code>confidentialTransferFrom(from, to, amount)</code>	Operator transfer using handle
<code>confidentialTransferFrom(from, to, amount, proof)</code>	Operator transfer with proof
<code>confidentialTransferAndCall(...)</code>	Transfer with ERC-1363 callback
<code>confidentialTransferFromAndCall(...)</code>	Operator transfer with callback

Example transfer:

```
import { fhevm } from 'hardhat';

// Create encrypted input
const encryptedInput = await fhevm
  .createEncryptedInput(tokenAddress, senderAddress)
  .add64(1000) // Amount to transfer
  .encrypt();

// Execute transfer
await token.connect(sender) ['confidentialTransfer(address,bytes32,bytes)'] (
  recipientAddress,
  encryptedInput.handles[0],
  encryptedInput.inputProof
);
```

Forced Transfer

Enforcers can move tokens from frozen addresses for regulatory compliance. Forced transfers can be performed even when the contract is deactivated.

```
function forcedTransfer(
  address from,           // Must be frozen
  address to,             // Must not be address(0)
  externalEuint64 encryptedAmount,
  bytes calldata inputProof
) public onlyRole(FORCED_OPS_ROLE) returns (euint64 transferred);
```

Requirements:

- The `from` address must be frozen
- The `to` address must not be `address(0)` -- use `forcedBurn()` for burning
- Can be performed even when the contract is deactivated

Note: This is intentionally stricter than standard CMTAT, which allows `forcedTransfer` on any address (frozen or not). CMTAT Confidential requires the address to be frozen first, creating an explicit audit trail (freeze event followed by forced transfer).

Forced Burn

Enforcers can burn tokens directly from frozen addresses without the holder's consent. This is the dedicated burn equivalent of `forcedTransfer`. Forced burns can be performed even when the contract is deactivated.

```
function forcedBurn(  
    address from,           // Must be frozen  
    externalEuint64 encryptedAmount,  
    bytes calldata inputProof  
) public onlyRole(FORCED_OPS_ROLE) returns (euint64 burned);
```

Requirements:

- The `from` address must be frozen
- Can be performed even when the contract is deactivated

Note: Same freeze requirement as `forcedTransfer` for consistency. The enforcer creates the encrypted input specifying how many tokens to burn.

Total Supply Visibility

By default the total supply is encrypted and inaccessible to third parties. Two mechanisms are available to open read access, gated by `SUPPLY_OBSERVER_ROLE` (observer list) or `SUPPLY_PUBLISHER_ROLE` (public disclosure).

Option 1 — Authorized observers (automatic, stays current) — `CMTATConfidential` only

Register addresses that will automatically receive ACL access to the total supply handle after every mint or burn:

```
// Grant SUPPLY_OBSERVER_ROLE to the compliance manager  
await token.grantRole(SUPPLY_OBSERVER_ROLE, complianceManager.address);  
  
// Grant SUPPLY_PUBLISHER_ROLE to the compliance manager (separate permission)  
await token.grantRole(SUPPLY_PUBLISHER_ROLE, complianceManager.address);  
  
// Register a regulator as a total supply observer (capped by  
maxSupplyObservers, default 10)  
await  
token.connect(complianceManager).addTotalSupplyObserver(regulatorAddress);  
  
// Remove an observer (stops future grants; past ACL grants are irrevocable)  
await  
token.connect(complianceManager).removeTotalSupplyObserver(regulatorAddress);  
  
// Inspect the current observer list and cap  
const observers = await token.totalSupplyObservers();  
const cap = await token.maxSupplyObservers(); // default 10
```

```
// Adjust the cap (DEFAULT_ADMIN_ROLE only; cannot go below current observer count)
await token.connect(admin).setMaxSupplyObservers(20);
```

Once registered, the observer can decrypt off-chain using the standard user-decryption flow:

```
const handle = await token.confidentialTotalSupply();
const supply = await fhevm.userDecryptEuint(FhevmType.euint64, handle, tokenAddress, observer);
```

Option 2 — Public disclosure (anyone, irrevocable per handle) — `CMTATConfidential` and `CMTATConfidentialLite`

Mark the current total supply handle as publicly decryptable. Any off-chain party can then request decryption via the Zama Relayer SDK without ACL access. After the next mint or burn, the new handle will not be publicly decryptable — call again if needed.

```
await token.connect(complianceManager).publishTotalSupply();
```

Mechanism	Availability	Access scope	Stays current after mint/burn
<code>addTotalSupplyObserver</code>	<code>CMTATConfidential</code> only	Specific registered addresses	Yes — re-granted automatically via <code>_afterMint/_afterBurn</code> hooks
<code>publishTotalSupply</code>	<code>CMTATConfidential</code> and <code>CMTATConfidentialLite</code>	<code>SUPPLY_PUBLISHER_ROLE</code>	No — must be called again after each mint/burn

Gas note: In `CMTATConfidential`, every mint or burn triggers `_updateTotalSupplyObserversACL()`, which iterates over all registered total supply observers and calls `FHE.allow()` for each one. Additionally, `_update` runs a chain of balance observer ACL grants. Keep both observer lists small to control gas costs per operation.

Pause / Unpause

Pause all transfers (requires `PAUSER_ROLE`):

```
function pause() public onlyRole(PAUSER_ROLE);
function unpause() public onlyRole(PAUSER_ROLE);
```

Freeze / Unfreeze

Freeze specific addresses (requires `ENFORCER_ROLE`):

```
function setAddressFrozen(address account, bool freeze) public
onlyRole(ENFORCER_ROLE);
function isFrozen(address account) public view returns (bool);
```

Warning: `ENFORCER_ROLE` holders must never freeze `address(0)`. The upstream CMTAT `EnforcementModule` does not guard against it. Direct holder transfers (`confidentialTransfer`, `confidentialTransferAndCall`) use `address(0)` as a synthetic spender in the freeze check, so freezing it would block **all** holder-initiated transfers, effectively acting as a global pause without holding `PAUSER_ROLE`. See [CMTAT](#) for a detailed analysis and the upstream fix proposal.

Deactivate Contract

Permanently deactivate the contract (requires `DEFAULT_ADMIN_ROLE`, contract must be paused):

```
function deactivateContract() public onlyRole(DEFAULT_ADMIN_ROLE);
```

Operator System

Operators can transfer tokens on behalf of holders using `confidentialTransferFrom`. Unlike ERC-20 allowances, operators have time-limited unlimited access.

```
// Set operator for 24 hours
const expirationTimestamp = Math.floor(Date.now() / 1000) + 86400;
await token.connect(holder).setOperator(operatorAddress, expirationTimestamp);

// Check operator status
const isOp = await token.isOperator(holderAddress, operatorAddress);
```

Warning: Setting an operator grants them access to transfer ALL your tokens during the approval period.

Decrypting Balances

Only the balance holder can decrypt their balance:

```
import { FhevmType } from '@fhevm/hardhat-plugin';

// Get encrypted balance handle
const balanceHandle = await token.confidentialBalanceOf(holderAddress);

// Decrypt (only works for the holder)
const balance = await fhevm.userDecryptEuint(
  FhevmType.euint64,
  balanceHandle,
  tokenAddress,
  holder // Signer must be the balance owner
);
```

Public Disclosure

Holders can publicly disclose amounts:

```
// Request disclosure (makes handle publicly decryptable)
function requestDiscloseEncryptedAmount(euint64 encryptedAmount) public;

// Finalize with decryption proof
function discloseEncryptedAmount(
    euint64 encryptedAmount,
    uint64 cleartextAmount,
    bytes calldata decryptionProof
) public;
```

Deployment

```
import { ethers } from 'hardhat';

const extraInfoAttributes = {
  tokenId: 'TOKEN-001',
  terms: {
    name: 'Terms Document',
    uri: 'https://example.com/terms',
    documentHash: ethers.ZeroHash,
  },
  information: 'Security token for XYZ',
};

const token = await ethers.deployContract('CMTATConfidential', [
  'My Token',           // name
  'MTK',                // symbol
  'https://example.com/metadata', // contractURI
  6,                    // decimals (choose per token, e.g. 0, 6, 8)
  adminAddress,         // admin with DEFAULT_ADMIN_ROLE
  extraInfoAttributes,  // token metadata
]);

// Grant roles
await token.grantRole(MINTER_ROLE, minterAddress);
await token.grantRole(BURNER_ROLE, burnerAddress);
await token.grantRole(PAUSER_ROLE, pauserAddress);
await token.grantRole(ENFORCER_ROLE, enforcerAddress); // freeze/unfreeze
addresses
await token.grantRole(FORCED_OPS_ROLE, enforcerAddress); // forced transfer /
forced burn
```

Choosing Decimals

Decimals are configurable at deployment for both `CMTATConfidential` and `CMTATConfidentialLite`. The constructor argument order is: `name`, `symbol`, `contractURI`, `decimals`, `admin`, `extraInfoAttributes`.

Warning: ERC-7984 balances use `euint64` (max `18,446,744,073,709,551,615` raw units). The maximum human-readable supply is `uint64 max / 10^decimals`:

Decimals	Max supply
----------	------------

Decimals	Max supply
0	$\sim 18.4 \times 10^{18}$ tokens
6	~ 18.4 trillion tokens (<i>recommended default</i>)
8	~ 184 billion tokens
18	~ 18 tokens

Values above **18 are rejected** at construction (`CMTAT_DecimalTooHigh`), because even a single token ($1 \times 10^{\text{decimals}}$ raw units) would overflow `uint64`. Values of 9 or more are technically valid but severely constrain the maximum supply — verify that $\text{expectedMaxSupply} \times 10^{\text{decimals}} \leq 18,446,744,073,709,551,615$ before deploying.

Note: FHE overflow is silent — a mint or transfer that exceeds `uint64` max will transfer `0` without reverting.

Dependencies

Package	Version
<code>@fhevm/solidity</code>	0.11.1
<code>@fhevm/hardhat-plugin</code>	0.4.2
<code>@zama-fhe/relayer-sdk</code>	0.4.1
<code>@openzeppelin/contracts</code>	5.6.1
<code>@openzeppelin/contracts-upgradeable</code>	5.6.1
Submodule	
OpenZeppelin Confidential Contracts	v0.4.0
CMTAT	v3.2.0

Project Structure

```

CMTAT-Confidential/
├── contracts/
│   ├── CMTATConfidentialBase.sol           # Abstract base (all
shared logic)
│   ├── CMTATConfidential.sol               # Full variant (+ total
supply visibility)
│   ├── CMTATConfidentialLite.sol           # Lite variant (smaller,
no total supply module)
│   └── modules/
│       ├── ERC7984MintModule.sol           # Mint with authorization
hook
│       ├── ERC7984BurnModule.sol           # Burn with authorization
hook
│       └── ERC7984EnforcementModule.sol     # Forced transfer and forced
burn

```

```

|   └─ ERC7984BalanceViewModule.sol           # Per-account balance
observers
|   └─ CMTATConfidentialVersionModule.sol       # CMTAT
Confidential version override (0.2.0)
|   └─ ERC7984PublishTotalSupplyModule.sol     # Public total supply
disclosure
|   └─ ERC7984TotalSupplyViewModule.sol        # Total supply observer list
(auto ACL)
└─ CMTAT/                                     # CMTAT submodule (compliance
modules)
└─ openzeppelin-confidential-contracts/        # OZ submodule (ERC7984)
└─ docs/
|   └─ fhe/                                   # Zama FHE documentation
|   └─ openzeppelin-confidential/             # OZ confidential docs
└─ test/
|   └─ CMTATConfidential.test.ts               # Full variant core
tests
|   └─ CMTATConfidentialLite.test.ts           # Lite variant core
tests (shared suite)
|   └─ ERC7984BalanceViewModule.test.ts        # Balance observer module
tests
|   └─ ERC7984PublishTotalSupplyModule.test.ts # Public disclosure module
tests
|   └─ ERC7984TotalSupplyViewModule.test.ts    # Total supply observer
module tests
|   └─ helpers/
|   └─ deploy.ts                             # Shared deploy helper + role
constants
|   └─ core-tests.ts                          # Shared Mocha test suite
|   └─ accounts.ts                            # Account impersonation utilities
└─ hardhat.config.js

```

FAQ

1. As an issuer, can I burn tokens from a token holder without their consent?

Answer: Yes, as an issuer with the `FORCED_OPS_ROLE`, you can burn tokens from any holder without their consent using the `forcedBurn()` function.

How it works:

1. First freeze the holder's address using `setAddressFrozen(holderAddress, true)` (requires `ENFORCER_ROLE`)
2. Use the `forcedBurn()` function to burn tokens directly from the frozen address (requires `FORCED_OPS_ROLE`)
3. This function can be performed even when the contract is deactivated
4. Only accounts with `FORCED_OPS_ROLE` can execute forced burns

Use cases for regulatory compliance:

- Court orders requiring asset seizure
- Sanctions compliance
- Error correction (e.g., tokens sent to wrong address)

Code example:

```
// Step 1: Freeze the holder's address
await token.connect(enforcer).setAddressFrozen(holderAddress, true);

// Step 2: Create encrypted input for the amount to burn
const encryptedInput = await fhevm
  .createEncryptedInput(tokenAddress, enforcerAddress)
  .add64(amountToBurn)
  .encrypt();

// Step 3: Force burn tokens from the frozen address
await token.connect(enforcer) ['forcedBurn(address,bytes32,bytes)'](
  holderAddress,                // from (must be frozen)
  encryptedInput.handles[0],    // encrypted amount
  encryptedInput.inputProof     // ZKPoK proof
);
```

Note: The regular `burn()` function requires `BURNER_ROLE` and will fail if the target address is frozen. For frozen addresses, use `forcedBurn()` instead (requires `FORCED_OPS_ROLE`). The `from` address must be frozen before calling `forcedBurn()`. Note that `forcedTransfer()` reverts if `to` is `address(0)` -- use `forcedBurn()` for burning.

Design choice: Standard CMTAT allows `forcedTransfer` and `forcedBurn` on any address (frozen or not). CMTAT Confidential intentionally requires the address to be frozen first, creating an explicit audit trail (freeze event followed by forced burn/transfer).

2. As a token holder, how do I transfer my tokens to another address?

Answer: Transfers in CMTAT Confidential use encrypted inputs to preserve confidentiality. Here's the complete process:

Step 1: Create an encrypted input

Encrypted inputs are data values submitted in ciphertext form, accompanied by **Zero-Knowledge Proofs of Knowledge (ZKPoKs)** to ensure validity without revealing the plaintext.

```
const encryptedInput = await fhevm
  .createEncryptedInput(tokenContractAddress, yourAddress)
  .add64(amount) // Amount to transfer (will be encrypted)
  .encrypt();
```

Step 2: Call the transfer function

```
await token.confidentialTransfer(
  recipientAddress,
  encryptedInput.handles[0], // externalEuint64 handle
  encryptedInput.inputProof  // ZKPoK proof
);
```

How validation works on-chain

1. **Input verification:** The `FHE.fromExternal()` function validates the ciphertext and ZKPoK
2. **Type conversion:** Converts `externalEuInt64` into `euInt64` for contract operations
3. **Balance check:** If balance is insufficient, transfer executes but transfers 0 (FHE doesn't reveal balance)

Alternative: Using an operator

You can authorize an operator to transfer on your behalf using time-limited approval:

```
const expirationTimestamp = Math.round(Date.now() / 1000) + 60 * 60 * 24; // 24
hours
await token.connect(holder).setOperator(operatorAddress, expirationTimestamp);

// Operator can now call confidentialTransferFrom
await token.connect(operator).confidentialTransferFrom(
  holderAddress,
  recipientAddress,
  encryptedAmount,
  inputProof
);
```

Important: Setting an operator allows them to transfer **all your tokens**. Carefully vet operators before approval.

Transfer requirements

- Your address must not be frozen
- The recipient address must not be frozen
- The contract must not be paused or deactivated

3. Can I deploy CMTAT Confidential contracts on Ethereum mainnet?

Answer: Yes - the Zama Protocol mainnet on Ethereum is now live.

How does it work?

Ethereum mainnet does not natively support FHE operations. The Zama Protocol uses a **coprocessor architecture** where the host chain (Ethereum) performs symbolic execution, and the actual FHE computations are performed off-chain by coprocessors. The infrastructure includes:

- **ACL (Access Control List):** Manages permissions for encrypted data
- **FHEVMExecutor (Coprocessor):** Performs encrypted computations off-chain
- **KMSVerifier:** Verifies decryption proofs from the Key Management System
- **InputVerifier:** Validates encrypted inputs and ZKPoKs

Where can you deploy?

Network	Chain ID	Status
Local development	31337	Supported (mock coprocessor via hardhat plugin)
Ethereum Sepolia	11155111	Supported (Zama testnet infrastructure)
Ethereum Mainnet	1	Live

Requirements for deployment

1. **Inherit from `ZamaEthereumConfig`**: This automatically configures coprocessor addresses:

```
import {ZamaEthereumConfig} from "@fhevm/solidity/config/ZamaConfig.sol";

contract MyToken is ERC7984, ZamaEthereumConfig {
    // Constructor automatically calls FHE.setCoprocessor()
}
```

2. **Target network must have Zama infrastructure**: The coprocessor contracts must be deployed and operational
3. **Users need compatible tools**: Client applications must use the Relay SDK to create encrypted inputs and request decryptions

4. Is it possible to also provide privacy for addresses?

Answer: Not in this implementation. CMTAT Confidential does **not** provide encrypted addresses.

This project keeps Ethereum addresses public and only encrypts amounts/balances (`euint64`).

Encrypted addresses are technically possible in fhEVM using the `eaddress` type.

How it works

The fhEVM library provides the `eaddress` type which encrypts Ethereum addresses. This enables:

- Hiding sender and recipient addresses in transfers
- Omnibus account patterns where multiple users share a single on-chain address

Implementation: Omnibus pattern

The OpenZeppelin Confidential Contracts library includes `ERC7984Omnibus` extension:

1. Uses a single omnibus address visible on-chain
2. Maintains encrypted mappings of actual user addresses to balances
3. Transfers happen between encrypted addresses within the omnibus

```
function confidentialTransferFromOmnibus(
    address omnibusFrom,
    address omnibusTo,
    externalEaddress externalSender,      // encrypted sender
    externalEaddress externalRecipient,  // encrypted recipient
    externalEuint64 externalAmount,
    bytes calldata inputProof
) public virtual returns (euint64);
```

Trade-offs

Benefit	Cost
Address privacy	Higher gas costs for encrypted address operations
Regulatory compliance (omnibus accounts)	More complex user experience
Institutional custody patterns	Requires trusted omnibus operators

Regulatory considerations

Hiding participant identities may conflict with AML/KYC requirements in some jurisdictions. Consider your compliance obligations before implementing address privacy.

5. Is the total supply public or private information?

Answer: The total supply is **private** (encrypted) by default in ERC7984.

Technical details

- The `_totalSupply` variable is stored as `euint64` (encrypted unsigned 64-bit integer)
- The `confidentialTotalSupply()` function returns an encrypted handle, not a plaintext value
- Access is controlled by the ACL (Access Control List)

```
// Returns an encrypted handle (euint64), not the actual value
function confidentialTotalSupply() public view returns (euint64);
```

How to grant access to the total supply

CMTAT Confidential provides two built-in mechanisms:

Option A — Authorized observers (stays current automatically) —

`ERC7984TotalSupplyViewModule`, `CMTATConfidential` only

Register specific addresses that automatically receive ACL access after every mint or burn:

```
token.addTotalSupplyObserver(regulatorAddress); // re-granted on every
mint/burn
token.removeTotalSupplyObserver(regulatorAddress); // stops future grants
```

Once registered, the observer decrypts using standard user-decryption — see [Decrypting Balances](#).

Option B — Public disclosure — `ERC7984PublishTotalSupplyModule`, available on both `CMTATConfidential` and `CMTATConfidentialLite`

Call `publishTotalSupply()` (requires `SUPPLY_PUBLISHER_ROLE`) to mark the current handle as publicly decryptable. Must be called again after each mint or burn since the handle changes. This will revert if total supply has never been initialized (no mint/burn yet).

```
token.publishTotalSupply();
```

Note: `publishTotalSupply()` reverts until the total supply handle is initialized (i.e., at least one mint or burn has occurred).

Internally this calls `FHE.makePubliclyDecryptable()`, which triggers the following **asynchronous three-step process**:

Public decryption — step by step

Public decryption splits work between on-chain and off-chain:

Step 1: On-chain - Mark as publicly decryptable

The contract sets the ciphertext handle's status as publicly decryptable, **globally and permanently** authorizing any entity to request its off-chain cleartext value.

```
FHE.makePubliclyDecryptable(confidentialTotalSupply());
```

Step 2: Off-chain - Request decryption from KMS

Any off-chain client can submit the ciphertext handle to the Zama Relay's Key Management System (KMS) using the Relay SDK (`@zama-fhe/relayer-sdk`).

```
const result = await fhevmInstance.publicDecrypt([totalSupplyHandle]);
// Returns:
// - clearValues: mapping of handles to decrypted values
// - abiEncodedClearValues: ABI-encoded byte string of all cleartext values
// - decryptionProof: cryptographic proof from the KMS
```

Step 3: On-chain - Verify and use the decrypted value

The caller submits the cleartext and decryption proof back to a contract function. The contract calls `FHE.checkSignatures`, which reverts if the proof is invalid.

```
FHE.checkSignatures(handlesList, abiEncodedCleartexts, decryptionProof);
// Now you can use the verified cleartext value
```

Important: The decryption proof is cryptographically bound to the specific order of handles passed in the input array.

ACL permissions

To access encrypted values, accounts need proper ACL permissions:

Function	Purpose
<code>FHE.allow(handle, address)</code>	Permanent access for specific address
<code>FHE.allowThis(handle)</code>	Shorthand for <code>FHE.allow(handle, address(this))</code> - allow current contract
<code>FHE.allowTransient(handle, address)</code>	Temporary access (current transaction only)
<code>FHE.makePubliclyDecryptable(handle)</code>	Allow anyone to decrypt off-chain
<code>FHE.isAllowed(handle, address)</code>	Check if an address has access to a ciphertext
<code>FHE.isSenderAllowed(handle)</code>	Check if <code>msg.sender</code> has access to a ciphertext

Why keep total supply private?

- Prevents market manipulation based on supply information
- Protects issuer's business information
- Consistent with the privacy-first design of confidential tokens

Note: If you need public total supply, implement a function that goes through the full decryption process and emits the result as an event. Consider the privacy implications carefully.

6. As an issuer, can I get the non-encrypted balance of a token holder?

Answer: Yes, but only through the asynchronous decryption process -- there is no direct way to "read" a plaintext balance.

How to decrypt a holder's balance

The issuer must have ACL permission on the holder's balance ciphertext. By default, only the balance holder and the contract itself have access. To allow the issuer to decrypt:

Option 1: Grant the issuer ACL access (on-chain)

The contract can grant ACL permission to the issuer's address on the holder's balance handle. This would need to be built into the contract logic (e.g., a function that grants the issuer access to a specific balance):

```
// Inside the contract, an admin function could grant access
function grantBalanceAccess(address holder, address viewer) public
onlyRole(DEFAULT_ADMIN_ROLE) {
    FHE.allow(confidentialBalanceOf(holder), viewer);
}
```

Once the issuer has ACL access, they can decrypt the balance off-chain:

```
const balanceHandle = await token.confidentialBalanceOf(holderAddress);
const balance = await fhevm.userDecryptEuInt(
    FhevmType.euInt64,
    balanceHandle,
    tokenAddress,
    issuer // Signer with ACL access
);
```

Option 2: Public decryption

The holder (or a contract function) can mark their balance as publicly decryptable, then anyone can request decryption through the three-step process (see FAQ #5).

Do I need a viewing key?

Zama's FHEVM does **not** use a "viewing key" model like some other privacy systems (e.g., Zcash). Instead, access is controlled through the **ACL (Access Control List)**:

Concept	Zama FHE Approach
Viewing key	Not applicable -- uses ACL permissions instead
Grant read access	<code>FHE.allow(handle, address)</code> grants permanent access to a specific ciphertext
Temporary access	<code>FHE.allowTransient(handle, address)</code> grants access for the current transaction only
Public access	<code>FHE.makePubliclyDecryptable(handle)</code> allows anyone to decrypt

The ACL model is more flexible than viewing keys: you can grant access per-ciphertext, per-address, and with different durations (permanent, transient, or public).

Can third-parties read balances?

Yes, if they are granted ACL permission. The contract logic decides who gets access.

The handle staleness problem

Every FHE arithmetic operation (mint, transfer, burn) produces a *new* ciphertext handle for the affected balance. A third party who was granted ACL access to an old handle loses the ability to read the balance the moment that handle is replaced. Access must therefore be re-granted after every update — it cannot be set once and forgotten.

CMTAT Confidential solves this with the `ERC7984ObserverAccess` extension. After every `_update`, the contract automatically calls `FHE.allow()` on the new balance handle for each registered observer, keeping their ACL access current without any manual intervention.

Two observer slots per holder

`ERC7984BalanceViewModule` (built on `ERC7984ObserverAccess`) provides two independent observer slots per address:

Slot	Set by	Typical use
------	--------	-------------

Slot	Set by	Typical use
Holder observer (<code>setObserver</code>)	The holder themselves	Personal wallet app, portfolio dashboard
Role observer (<code>setRoleObserver</code> , requires <code>OBSERVER_ROLE</code>)	The issuer / compliance team	Regulator, auditor, compliance tool

Both slots receive `FHE.allow()` on every balance update automatically.

Observer ACL scope: balance *and* transfer amount

On every `_update`, observers are granted ACL access to **two** ciphertext handles:

1. The account's new **balance** handle — so the observer can read the current balance at any time
2. The **transferred amount** handle — so the observer can reconstruct individual transaction amounts

This is intentional design for regulatory compliance: a compliance observer needs transfer-level granularity, not just balance snapshots. An observer set via `setRoleObserver` should therefore be treated as having access to individual transaction amounts for the account they are observing.

Decrypting as an observer

Once access is granted the observer can decrypt off-chain through the standard user-decryption flow:

```
// Read the current handle
const handle = await token.confidentialBalanceOf(holderAddress);

// Decrypt — observer must have ACL access on that handle
const balance = await fhevm.userDecryptEuInt(
    FhevmType.euInt64,
    handle,
    tokenAddress,
    observer // signer with ACL access
);
```

ACL access is permanent and cannot be revoked

`FHE.allow()` is a one-way operation — once an observer is granted access to a handle, that access cannot be removed. Removing an observer with `setObserver` / `setRoleObserver` only stops *future* grants: the observer retains read access to all handles they were allowed on before removal.

Common patterns

Party	How access is granted
The holder themselves	Automatically by the contract on every transfer/mint
Regulatory observer	Issuer calls <code>setRoleObserver(holder, regulatorAddress)</code>
Personal observer	Holder calls <code>setObserver(holderAddress, walletAppAddress)</code>

Party	How access is granted
Auditor (temporary)	Admin calls <code>FHE.allowTransient()</code> during the audit transaction

7. Can the issuer compute the inputProof for operations if the issuer is not the token holder?

Answer: Yes. The `inputProof` (Zero-Knowledge Proof of Knowledge) is generated by whoever creates the encrypted input, not by the token holder.

How encrypted inputs work

When any party calls a function that requires an encrypted amount (mint, burn, forcedBurn, forcedTransfer), they:

1. **Choose the plaintext amount** they want to encrypt
2. **Generate the encrypted input** using the FHEVM client library
3. **The library produces** a ciphertext handle + a ZKPoK proof

The ZKPoK proves that the caller **knows** the plaintext value inside the ciphertext, without revealing it. It does not prove anything about token ownership or balances.

Example: Issuer burns tokens from a holder

```
// The enforcer (issuer) creates the encrypted input themselves
const encryptedInput = await fhevm
  .createEncryptedInput(tokenAddress, enforcerAddress) // enforcer's address,
  NOT holder's
  .add64(amountToBurn)
  .encrypt();

// The enforcer calls forcedBurn with their own proof
await token.connect(enforcer) ['forcedBurn(address,bytes32,bytes)'](
  holderAddress,                // target to burn from
  encryptedInput.handles[0],     // enforcer's encrypted amount
  encryptedInput.inputProof      // enforcer's proof
);
```

Key points

Question	Answer
Who generates the inputProof?	The caller of the function (e.g., the enforcer/issuer)
Does the holder need to participate?	No -- forced operations don't require holder involvement
Is the proof tied to the holder's balance?	No -- it proves knowledge of the encrypted input value, not the balance

Question	Answer
Can the issuer specify any amount?	Yes, but if the amount exceeds the holder's balance, FHE transfers/burns 0 silently

The `inputProof` is tied to the **caller's address** and the **contract address** (passed to `createEncryptedInput`), not to the token holder. This is what enables administrative operations like `forcedBurn` and `forcedTransfer` without the holder's participation.

Glossary

Term	Definition
FHE (Fully Homomorphic Encryption)	Cryptographic scheme that enables arbitrary computations directly on ciphertext without ever decrypting it. The result, once decrypted, is identical to what would have been produced on the plaintext. It is the core primitive behind confidential balances and transfers in CMTAT Confidential.
euint64	Encrypted unsigned 64-bit integer — the on-chain type used to store confidential balances and transfer amounts. It is a pointer to a ciphertext managed by the Zama coprocessor network, not a raw encrypted value. Maximum representable value is $\sim 18.4 \times 10^{18}$.
externalEuint64	The user-facing form of an encrypted 64-bit integer: a ciphertext handle produced by the client library and submitted alongside a ZKPoK. Converted to <code>euint64</code> on-chain via <code>FHE.fromExternal()</code> after the proof is verified.
ZKPoK (Zero-Knowledge Proof of Knowledge)	A cryptographic proof that the submitter knows the plaintext inside an encrypted input, without revealing that plaintext. Required for every encrypted input to prevent malleability and replay attacks. Verified on-chain by the InputVerifier contract.
Ciphertext Handle	A 32-byte pointer returned by every FHE operation (e.g., <code>euint64</code>). It references the actual ciphertext stored and computed by the coprocessor network, not the ciphertext itself. Arithmetic on handles triggers off-chain coprocessor computation and produces new handles.
ACL (Access Control List)	On-chain permission registry that tracks which addresses may decrypt a given ciphertext handle. Access is granted with <code>FHE.allow()</code> (permanent) or <code>FHE.allowTransient()</code> (current transaction only). Without ACL access, a party cannot request decryption of a handle.
Coprocessor (FHEVMExecutor)	Off-chain network of nodes that performs the actual FHE computations. When a Solidity contract calls an FHE operation, the host chain emits an event; coprocessors pick it up, compute the result, and make the new ciphertext available.

Term	Definition
KMS (Key Management System)	Zama's key management infrastructure that holds the FHE private key in a distributed manner using MPC. Decryption requests are sent to the KMS, which returns a decrypted value together with a cryptographic proof that can be verified on-chain.
MPC (Multi-Party Computation)	Threshold cryptography protocol used by the KMS so that no single node ever holds the complete FHE private key. Decryption only succeeds when a quorum of KMS nodes cooperates, preventing any single point of compromise.
Symbolic Execution	The on-chain execution model for FHE operations: the EVM does not perform the actual FHE computation — it only records an intent (emits an event with a new handle). The coprocessor network executes the computation off-chain asynchronously.
ERC-7984	The Confidential Fungible Token standard (drafted by OpenZeppelin). It defines the interface for tokens with encrypted balances and transfer amounts, including the operator system, disclosure mechanism, and ACL-based access patterns used by CMTAT Confidential.
Operator	An address authorized by a token holder to call <code>confidentialTransferFrom</code> on their behalf. Unlike ERC-20 allowances, operators are granted time-limited <i>unlimited</i> access — they may transfer any amount until the approval timestamp expires.
Observer	An ACL-authorized third party (e.g., a regulator or auditor) granted read access to one or more encrypted balance handles. Implemented via <code>ERC7984ObserverAccess</code> : the contract grants <code>FHE.allow()</code> on balances affected by each transfer, giving the observer a continuously updated view.

References

- [CMTAT - Capital Markets and Technology Association Token Standard](#)
- Openzeppelin
 - [OpenZeppelin Confidential Contracts](#)
 - [ERC-7984 Specification](#)
- Zama
 - [Zama Protocol Litepaper](#)
 - [Zama FHEVM Documentation](#)
 - [Zama FHE Types](#)
 - [Encrypted Inputs](#)
 - [Access Control List \(ACL\)](#)
 - [Decryption](#)
- Part of this project was carried out with the help of [Claude Code](#) and [Codex](#)

License

This project is licensed under the MPL-2.0 License.